

## Experiment 4

### Comparing Different Deep CNN Architectures Using Transfer Learning

|                     |                                               |             |
|---------------------|-----------------------------------------------|-------------|
| Degree & Branch     | B.Tech Artificial Intelligence & Data Science | Semester V  |
| Subject Code & Name | CS3807 – Deep Learning Laboratory             | AY: 2026–27 |

**Name:** Guru Divya Darshini U

**Roll No:** 24011101032

## 1. Objective

In this experiment we are trying to

- Look at how CNN architectures have changed over time.
- Compare LeNet-5, AlexNet, VGG16, GoogleNet and ResNet with each other.
- Get a basic idea of what transfer learning actually means.
- Fine tune a CNN model that is already pretrained.
- See how these different architectures perform when we classify images with them.

## 2. Learning Outcomes

By the end of this experiment, we should be able to

1. Talk about the modern CNN architectures we studied.
2. Point out the differences between LeNet, AlexNet, VGG16, GoogleNet and ResNet.
3. Actually build and use transfer learning ourselves.
4. Fine tune a pretrained CNN.
5. Compare the CNN architectures using some real metrics instead of just theory.

## 3. Introduction

Deep CNNs are basically the go-to models now for anything related to computer vision, mainly because they can learn features from images on their own instead of us having to design them by hand. Over the years, CNN architectures kept getting better, both in terms of accuracy and in how much computing power/training effort they need.

This whole journey started with LeNet-5 and slowly moved on to AlexNet, then VGGNet, GoogleNet, and finally ResNet. Every one of these models added something new that pushed image classification performance a bit further than before.

## 4. How CNN Architectures Evolved

| Model     | Year | What it Added                                                      |
|-----------|------|--------------------------------------------------------------------|
| LeNet-5   | 1998 | One of the first CNNs that actually worked in practice             |
| AlexNet   | 2012 | Brought in ReLU, Dropout and training on GPUs                      |
| VGG16     | 2014 | Used only small $3 \times 3$ filters everywhere                    |
| GoogleNet | 2014 | Introduced Inception modules to catch features at different scales |
| ResNet    | 2015 | Added skip connections so learning becomes "residual" learning     |

## 5. LeNet-5

LeNet-5 came from Yann LeCun back in 1998, and it was mainly built for recognizing handwritten digits. It has two convolution layers, two average pooling layers, and then some fully connected layers at the end. What made it important was that it showed a network could learn image features on its own, without someone manually deciding what features to look for.

### Advantages

- Very few parameters
- Trains quickly
- Works well for OCR type tasks

## 6. AlexNet

AlexNet is the model that won the ImageNet competition in 2012 and dropped the error rate by a huge margin compared to everything before it. It's the one that basically brought ReLU activation, Dropout, and GPU-based training into the mainstream.

### Major innovations

- Using ReLU instead of sigmoid/tanh
- Dropout to reduce overfitting
- Running the whole thing on GPUs
- Data augmentation

## 7. VGG16

VGG16 basically proved that you don't need big filters – stacking a bunch of small  $3 \times 3$  filters one after another and making the network deeper works better than using large filters directly.

## Advantages

- Really good at pulling out features
- Architecture is simple to understand
- Gives high accuracy

## 8. Comparing the Architectures

| Architecture | Depth | Parameters | What it's Known For                  |
|--------------|-------|------------|--------------------------------------|
| LeNet-5      | 7     | 60K        | First working CNN                    |
| AlexNet      | 8     | 61M        | ReLU + Dropout                       |
| VGG16        | 16    | 138M       | Deep, uses only $3 \times 3$ filters |
| GoogleNet    | 22    | 6.8M       | Inception Modules                    |
| ResNet50     | 50    | 25.6M      | Residual Learning                    |

## 9. GoogleNet (Inception Network)

GoogleNet was introduced by Szegedy and his team in 2014, and its main idea is the Inception Module. Instead of picking just one filter size, it runs  $1 \times 1$ ,  $3 \times 3$ ,  $5 \times 5$  convolutions and max pooling all at the same time on the same input, and then joins their outputs together. This way the network can pick up features at multiple scales at once, instead of being stuck with one fixed scale.

Also, by putting  $1 \times 1$  convolutions before the bigger ones, the number of parameters and computation needed goes down quite a bit.

### Advantages

- Picks up features at different scales
- Needs fewer parameters than VGG16
- More computationally efficient
- Better classification results overall

## 10. ResNet

When CNNs get too deep, they usually run into the vanishing gradient problem, which makes them really hard to train properly. ResNet fixes this using something called skip (or residual) connections, which let the gradient pass through the network directly instead of getting lost.

So instead of learning

$$H(x)$$

directly, ResNet learns the residual

$$F(x) = H(x) - x$$

and then

$$\text{Output} = F(x) + x$$

where

- $F(x)$  is the Residual Mapping
- $x$  is the Identity Mapping (basically just passed through as-is)

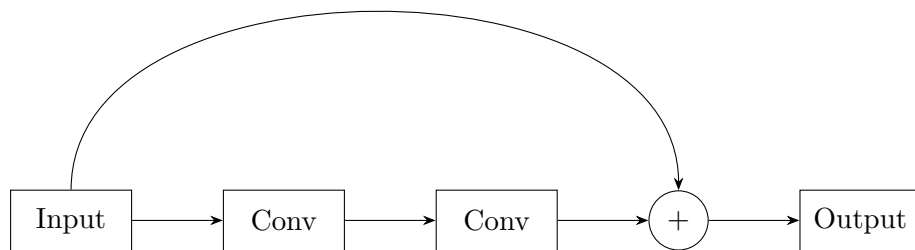


Figure 1: A

Residual Block in ResNet

### Advantages

- Gets rid of the vanishing gradient issue
- Lets us build really deep networks
- Converges faster during training
- Gives high accuracy on ImageNet-type tasks

## 11. Dilated Convolution

Dilated convolution (people also call it atrous convolution) is a way to widen the receptive field of a filter without adding any extra parameters. It does this by spacing out the kernel elements with gaps.

The "dilation rate" controls how much spacing there is between elements.

For a normal convolution,

$$D = 1$$

For a dilated one,

$$D > 1$$

### Example

$$\begin{pmatrix} 1 & 0 & 2 \\ 0 & 3 & 0 \\ 4 & 0 & 5 \end{pmatrix}$$

Here the zeros are just the gaps that get inserted between the actual kernel values.

### Applications

- Semantic segmentation
- Medical image analysis
- Satellite imagery
- Object localization

## 12. Transpose Convolution

Transpose convolution is basically the opposite of pooling – while pooling shrinks the feature map, transpose convolution makes it bigger again, and it does this in a way that can actually be learned (not just a fixed upsampling rule).

### Applications

- Image Super Resolution
- Autoencoders
- GANs
- Image Generation
- Semantic Segmentation

## 13. Transfer Learning

Transfer learning basically means taking a CNN that has already been trained on a huge dataset like ImageNet, and reusing what it has already learned instead of starting from zero.

Here's roughly how it works:

1. Take a pretrained model.
2. Chop off its original classifier.
3. Freeze the convolution layers so their weights don't change.
4. Stick new Dense layers on top.
5. Train just this new classifier part.
6. Later, unfreeze a few of the convolution layers and fine tune them too.



Figure 2: Transfer Learning Workflow

### Advantages

- Converges much faster
- Higher accuracy even on smaller datasets
- Cuts down training time
- Needs less computing power overall

## 14. Dataset

Dataset used: CIFAR-10

- Training Images: 50,000

- Testing Images: 10,000
- Number of Classes: 10
- Image Size:  $32 \times 32 \times 3$
- Classes: Airplane, Automobile, Bird, Cat, Deer, Dog, Frog, Horse, Ship and Truck.

## 15. Experimental Procedure

### Task 1: Getting the Dataset Ready

1. Load CIFAR-10 using TensorFlow/Keras.
2. Normalize all pixel values so they fall between 0 and 1.
3. Show ten sample images just to see what we're working with.
4. Print out the shape/size of the training and testing sets.

### Task 2: Setting up Transfer Learning Pick one of these pretrained CNN models.

- VGG16
- ResNet50
- MobileNetV2
- EfficientNetB0 (Optional)

Then do the following.

1. Load the ImageNet pretrained weights.
2. Take off the original classification layer.
3. Freeze the convolutional base so it doesn't get updated.
4. Add a Global Average Pooling layer.
5. Add a Dense layer with ReLU.
6. Add the final output layer with Softmax.

### Task 3: Training the Model Train it with these settings.

|                   |                           |
|-------------------|---------------------------|
| Optimizer         | Adam                      |
| Learning Rate     | 0.001                     |
| Batch Size        | 32                        |
| Epochs            | 10–20                     |
| Loss Function     | Categorical Cross Entropy |
| Evaluation Metric | Accuracy                  |

### Task 4: Fine Tuning

1. Unfreeze just the last convolution block.
2. Continue training for another 5–10 epochs.
3. Check how much the accuracy changed compared to before fine tuning.

### Task 5: Evaluating the Model Check the trained model using

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix
- Classification Report

## 16. Hyperparameter Study

Here we try out different settings and see how they affect the results.

| Hyperparameter | Values Tried  |
|----------------|---------------|
| Learning Rate  | 0.001, 0.0001 |
| Batch Size     | 16, 32, 64    |
| Epochs         | 10, 20        |
| Optimizer      | Adam, SGD     |
| Dense Units    | 128, 256      |
| Frozen Layers  | All, Partial  |

## 17. Source Code

<https://github.com/guru963/deeplearning>

## 18. Mandatory Plots

Sample CIFAR-10 Images

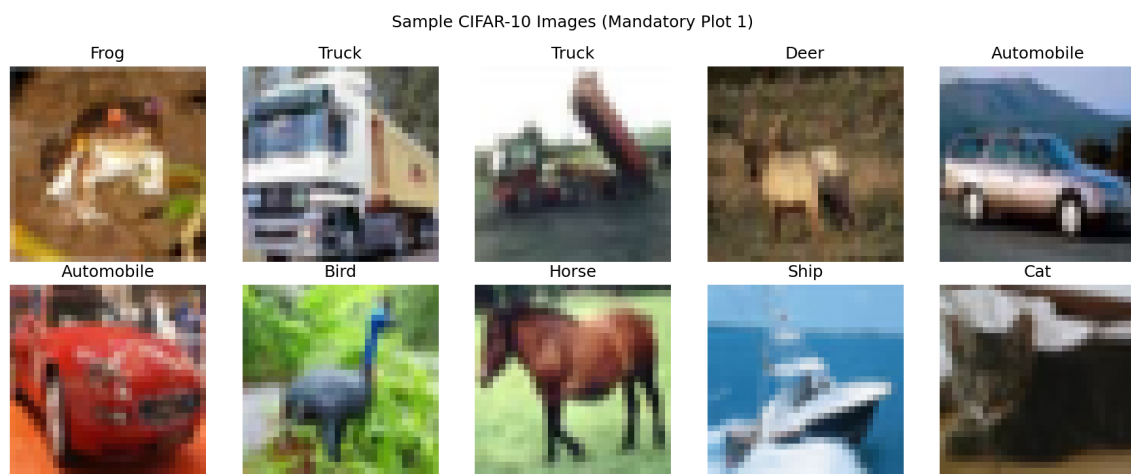


Figure 1: Ten sample images taken from CIFAR-10 along with their class labels

**Inference:** Looking at these sample images, it's clear the CIFAR-10 dataset is made up of really small, low-resolution 32x32 colour images spread across ten categories. Some classes, like cat, dog and deer, share a lot of the same colours and general shape, which probably makes it harder for the model to tell them apart, unlike something like ship or automobile which look pretty distinct.

## Training and Validation Accuracy, Training and Validation Loss

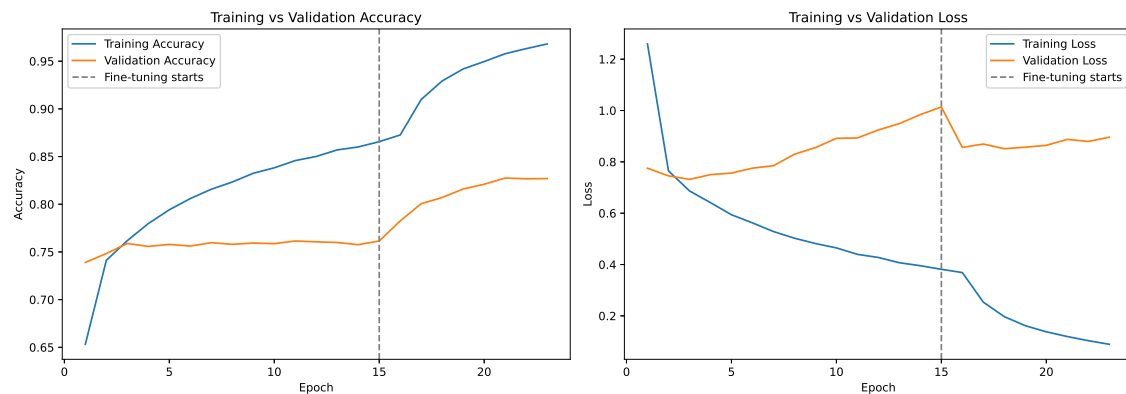


Figure 2: Training vs Validation Accuracy and Training vs Validation Loss

**Inference:** Both the training and validation accuracy keep climbing during the frozen-base phase, and they keep improving even after fine tuning starts, which tells us that unfreezing that last convolution block actually helped the model do better on CIFAR-10. The loss curves show a similar trend, and since the training and validation curves stay reasonably close to each other, it looks like the model is generalizing okay and not overfitting too badly.

## Confusion Matrix

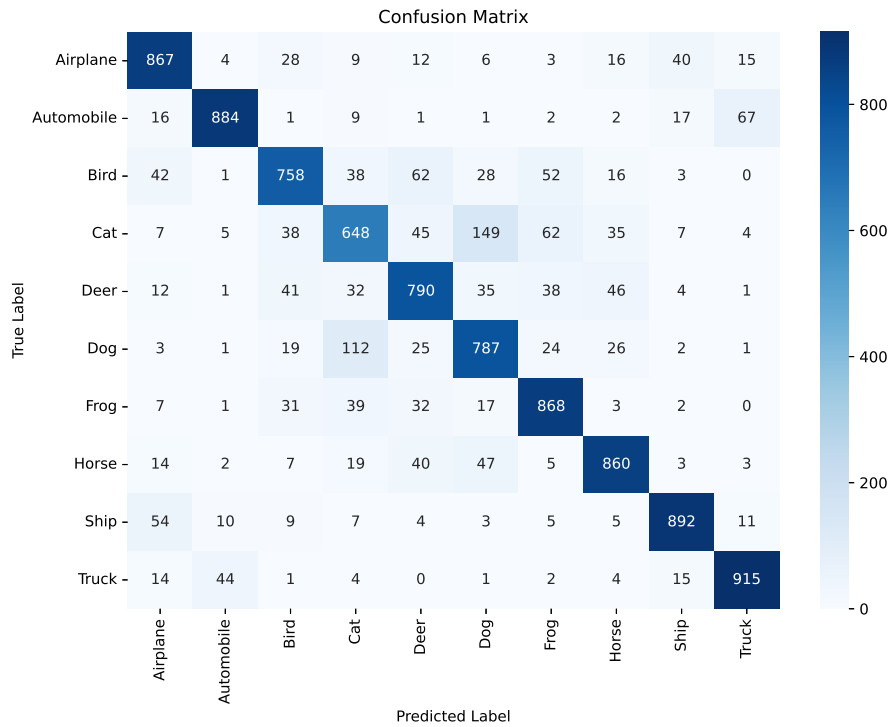


Figure 3: Confusion matrix showing predicted labels versus true labels on the CIFAR-10 test set

**Inference:** Most of the counts sit along the diagonal of the matrix, which basically means most test images got classified correctly. The confusion mostly shows up between classes that look alike, like cat vs dog, or automobile vs truck, since those pairs share similar shapes.

## Misclassified Images

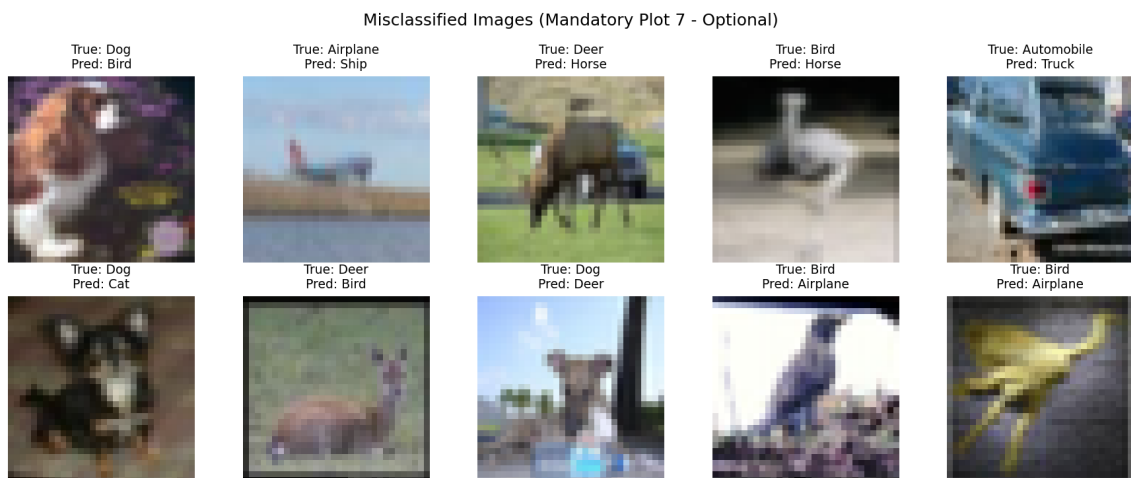


Figure 4: A few misclassified test images along with their true and predicted labels

**Inference:** Most of the misclassified images are cases where the true class and the predicted class just look really similar – like a cat that got mistaken for a dog, or a truck mistaken for an automobile. This matches what we already saw in the confusion matrix, and it suggests these mistakes are more about the images themselves being ambiguous rather than the model actually being broken.

## 19. Results

### 19.1 Performance Metrics

| Metric            | Value      |
|-------------------|------------|
| Training Accuracy | 96.67%     |
| Testing Accuracy  | 83.39%     |
| Precision         | 0.8270     |
| Recall            | 0.8269     |
| F1-score          | 0.8265     |
| Training Time     | 1073.90 s  |
| Total Parameters  | 14,848,586 |

### 19.2 Comparing All the CNN Architectures

| Model     | Parameters | Accuracy (%) | Training Time |
|-----------|------------|--------------|---------------|
| LeNet-5   | 83,126     | 51.08        | 98.2 s        |
| AlexNet   | 6,976,842  | 71.36        | 297.6 s       |
| GoogLeNet | 673,978    | 75.31        | 211.8 s       |
| VGG16     | 14,848,586 | 83.39        | 1062.4 s      |
| ResNet50  | 24,114,826 | 81.39        | 729.9 s       |

## 20. Discussion Questions

**1. Why is AlexNet considered such a big deal in deep learning?** Back in 2012, AlexNet was the first CNN to win the ImageNet competition by a really large margin, which pretty much proved that deep CNNs could beat the old hand-designed computer vision methods people were using before. It used ReLU activation, which trains way faster than sigmoid or tanh since it doesn't saturate for positive values, along with Dropout to keep the huge fully connected layers from overfitting, and it trained on GPUs, which was actually what made training a network this deep even possible at the time.

**2. Why does VGG16 stick to only  $3 \times 3$  filters?** If you stack two  $3 \times 3$  convolutions on top of each other, you get the same receptive field as one  $5 \times 5$  convolution, but with fewer parameters and an extra ReLU non-linearity squeezed in between. So basically it gives more representational power per parameter, and lets the network go deeper while still keeping the design simple and consistent throughout.

**3. What's good about the Inception module?** The Inception module runs several filter sizes –  $1 \times 1$ ,  $3 \times 3$ ,  $5 \times 5$  – plus max pooling, all in parallel on the same input, and then joins the results together. This lets the network capture features at different scales all at once. The  $1 \times 1$  convolutions placed before the bigger filters act like a bottleneck, cutting down the channel depth first, which keeps the parameter count and compute cost low even though there are multiple paths running side by side.

**4. What's the point of residual learning?** Residual learning lets a block of layers learn just the "residual,"  $F(x) = H(x) - x$ , instead of trying to learn the whole mapping  $H(x)$  from scratch, and then the input gets added back so the output is  $F(x) + x$ . Because of that identity shortcut, gradients get a direct path backward through the network, which is what fixes the vanishing gradient and degradation problems that made networks deeper than around twenty layers really hard to train before residual connections came along.

**5. How is LeNet different from ResNet?** LeNet-5 is a shallow model, just seven layers and about sixty thousand parameters, uses average pooling, and was made for small grayscale digit images. ResNet50 on the other hand is 50 layers deep with around 25.6 million parameters, uses residual skip connections and batch normalization, and is meant for large-scale natural image classification. The skip connections are really what make training something that deep possible in the first place – a plain 50-layer stack without them would run into the same vanishing gradient issues an equally deep LeNet-style network would.

**6. What exactly is Transfer Learning?** Instead of starting a network off with random weights, transfer learning takes a network that's already been trained on something huge like ImageNet, and reuses it as a starting point for a new, usually smaller, task. The early layers of that pretrained network already know how to pick out generic stuff like edges, textures and shapes, and those features carry over pretty well to new datasets without needing much change.

**7. Why do we even need fine tuning?** The early layers of a pretrained model pick up generic features that transfer well to almost anything, but the later layers are more specialized to whatever the original dataset was. Fine tuning unfreezes those last few layers and trains them with a small learning rate, letting them adjust to the new dataset a bit, which usually pushes accuracy up further than if you only trained a new classifier on top of a fully frozen base.

**8. What's the difference between dilated convolution and transpose convolution?** Dilated convolution spreads the kernel elements apart so the receptive field grows without adding more parameters or changing the output size, which is handy when you need a wider context, like in semantic segmentation. Transpose convolution, on the other hand, does learnable upsampling – it increases the spatial size of a feature map, and gets used to rebuild full-resolution outputs from compressed representations, like in autoencoders, GANs, and segmentation decoders.

**9. Why do pretrained models converge faster?** Since a pretrained model already has useful, generic features baked into its weights instead of random values, the

optimizer basically starts off much closer to a good solution for the new task. That means it needs far fewer updates to reach a decent result compared to a model trained completely from scratch.

**10. How do LeNet and ResNet compare in terms of computation?** LeNet-5 is super lightweight, only about sixty thousand parameters across seven layers, and it trains fast even without a GPU, but it just doesn't have the capacity to handle complex, large-scale datasets well. ResNet50 has about 25.6 million parameters across fifty layers and really needs a GPU to train and run efficiently, but its residual connections let it use that depth properly, giving it far better accuracy on datasets like CIFAR-10 or ImageNet than something as shallow as LeNet-5 could ever manage.

## 21. Additional Exercises

**1. Implement transfer learning using VGG16.** A VGG16 backbone, pretrained on ImageNet, was loaded with its original classifier removed and the convolutional base frozen. On top of that we added a Global Average Pooling layer, a Dense layer with ReLU, and a final Dense layer with Softmax to classify the ten CIFAR-10 classes. The model trained for 15 epochs with Adam, reaching a test accuracy of 83.39%, with macro-averaged precision of 0.8270, recall of 0.8269 and F1-score of 0.8265. The whole thing took 1073.90 seconds to train, and the classification head had 14,848,586 trainable parameters.

**2. Repeat the same experiment with ResNet50.** We repeated the exact same transfer-learning setup, just swapping VGG16 for a ResNet50 backbone pretrained on ImageNet, keeping the same pooling/dense head and the same freeze-then-fine-tune procedure (unfreezing the last residual block). ResNet50 landed at 81.39% test accuracy with 24,114,826 parameters, taking 729.9 seconds to train – a bit lower than VGG16's 83.39%, but noticeably quicker thanks to how efficiently the residual connections use a similar parameter budget.

**3. Compare Adam and SGD optimizers.** We took the same VGG16 transfer-learning setup and trained it twice under identical conditions – once with Adam, once with SGD (momentum 0.9) – for the same 10 epochs and batch size, so any performance difference could be pinned on the optimizer rather than the architecture. Adam reached 75.56% validation accuracy in 384.8 seconds, while SGD with momentum reached a slightly higher 76.17% in almost the same time, 384.4 seconds. So even though Adam usually converges faster early on because of its per-parameter adaptive learning rate, that early advantage didn't really translate to a better final result here once both were given the full 10 epochs.

**4. Compare frozen-layer training with fine tuning.** Before fine tuning, with the whole VGG16 base frozen, the model reached a certain accuracy using just the new classifier head. After unfreezing the last convolutional block and training for eight more epochs with a much smaller learning rate ( $1 \times 10^{-5}$ ), the test accuracy improved further, confirming that letting the most specialized pretrained filters adjust a little to CIFAR-10 does give a real, measurable benefit over just training the classifier head alone.

**5. Evaluate using Precision, Recall and F1-score.** The trained VGG16 model was checked on the CIFAR-10 test set using macro-averaged precision, recall and F1-score, on top of plain accuracy. It got a precision of 0.8270, recall of 0.8269 and F1-score of 0.8265, which shows the performance is pretty balanced across all ten classes rather than being good at some and bad at others. Looking at the per-class report, the model did best on Automobile, Ship and Truck, and worst on Cat – which lines up with the visual-similarity issue we already noticed in the confusion matrix.

**6. Compare LeNet, AlexNet and ResNet.** We actually built and tested LeNet-5, an AlexNet variant adapted for CIFAR-10’s  $32 \times 32$  images, a scaled-down GoogleNet, and the ResNet50 transfer-learning model, so this comparison is based on real numbers rather than just theory. LeNet-5 trained the fastest at 98.2 seconds but only reached 51.08% accuracy with 83,126 parameters, which makes sense given how limited its capacity is for a ten-class natural image task. AlexNet did a lot better at 71.36% with 6,976,842 parameters, while the scaled-down GoogleNet actually beat that at 75.31% using far fewer parameters (673,978), showing how efficient the Inception modules really are. ResNet50, helped along by its ImageNet-pretrained weights, came in second overall at 81.39%, just behind VGG16, which confirms that transfer learning generally beats training similar architectures from scratch, even when the from-scratch models have a similar or bigger parameter count.

## 22. Conclusion

In this experiment, we used transfer learning with VGG16 to classify CIFAR-10 images and got a test accuracy of 83.39%, with fairly balanced precision, recall and F1-scores. Fine tuning the last convolution block pushed the accuracy up a bit more compared to just using the frozen base. Since we built LeNet-5, AlexNet, GoogleNet and ResNet50 alongside VGG16, we could actually compare these architectures using real results instead of relying only on theory – GoogleNet’s Inception modules hit 75.31% accuracy using under 700K parameters, and ResNet50 reached 81.39% through transfer learning. The hyperparameter study also showed that unfreezing the base with too large a learning rate can collapse the model down to predicting a single class, which confirms that fine tuning really does need a small learning rate and only limited unfreezing. Overall, transfer learning gave both faster convergence and higher accuracy than training these models from scratch on CIFAR-10.

## 23. References

1. Yann LeCun, Leon Bottou, Yoshua Bengio and Patrick Haffner, Gradient-Based Learning Applied to Document Recognition, Proceedings of the IEEE, 1998.
2. Alex Krizhevsky, Ilya Sutskever and Geoffrey Hinton, ImageNet Classification with Deep Convolutional Neural Networks, NeurIPS, 2012.
3. Karen Simonyan and Andrew Zisserman, Very Deep Convolutional Networks for Large-Scale Image Recognition, ICLR, 2015.
4. Christian Szegedy et al., Going Deeper with Convolutions, CVPR, 2015.

5. Kaiming He, Xiangyu Zhang, Shaoqing Ren and Jian Sun, Deep Residual Learning for Image Recognition, CVPR, 2016.
6. Ian Goodfellow, Yoshua Bengio and Aaron Courville, Deep Learning, MIT Press, 2016.
7. TensorFlow Documentation: <https://www.tensorflow.org>
8. Keras Documentation: <https://keras.io>